



PROGRAMACIÓN

1º DAM

RETO 3



ENUNCIADO

L@s pesad@s de segundo de DAW, que casi no tienen tiempo para nada, nos han vuelto a pedir ayuda. En este caso en la interfaz de inicio de sesión se han planteado comprobar si el usuario introduce bien un correo electrónico. Para ello nos plantean tratar las excepciones en la entrada de datos. Vamos a recoger un tipo de dato String con un formato de correo electrónico: hola@damiansu.com.

Quieren controlar las siguientes premisas, en el caso de que se no se cumplan lanzarán un error con el mensaje pertinente:

- Debe de tener el símbolo @
- Que acabe por un dominio válido, en nuestro caso será .com y .es
- El correo electrónico debe empezar siempre por una letra nunca por un número.

RESOLUCIÓN

> Se realiza un programa de Java empleando el IDE NeatBens, que contendrá en el mismo paquete dos clases/archivos:

> 1 En primer lugar creamos una **clase** que llamamos **ExcepcionEmail** para gestionar los errores de formato en el correo introducido.

```
package reto3;

public class ExcepcionEmail extends Exception {

    //Constructores

    //1 --> Por defecto: construye pasando un mensaje predefinido,
    //por defecto, a la clase padre (Exception)
    public ExcepcionEmail() {
        super("El correo electrónico introducido no es válido");
    }

    //2 --> Construye pasando un mensaje específico
    //recibido en una variable tipo String
    public ExcepcionEmail(String mensaje) {
        super(mensaje);
    }

}
```

> Se emplea herencia para que la clase herede los métodos de la clase genérica **Exception**, mediante la palabra reservada **extends**.

> Se disponen **dos constructores** para la clase:

> Uno vacío, **por defecto**, que construye con un mensaje interno predefinido en el código ("El correo electrónico introducido no es válido") al pasárselo a la clase padre mediante **super()**.

> Un **segundo constructor**, que construye tomando como parámetro el mensaje específico que se le pasa al llamarlo (variable **mensaje**, tipo **String**) y que a su vez lo pasa internamente a la clase padre Exception para que construya con el mediante **super(mensaje)**.



> 2 En segundo lugar, creamos una **clase** que Principal donde definimos:

> Un bloque de código para pedir por pantalla el correo a validar como String y una llamada a un **método** que llamamos **validarCorreo** al que se le pasa el correo a validar en formato String y que contiene la gestión de excepciones. Trabajando de esta forma se puede separar el flujo de código principal de la gestión de errores y tenerlo más ordenado y de una manera más limpia, sobre todo ante la posibilidad de que se realicen más operaciones en el flujo de código de la clase Principal.



```
package reto3;

import java.util.Scanner;

public class Principal {

    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        System.out.println("--Introduzca un email con formato válido--");
        System.out.println("-----");
        System.out.println("""
            - Contendrá @
            - Dominios válidos: .com y .es
            - Comenzará por una letra
            """);
        System.out.println("");
        System.out.print("Email: ");

        //Toma por teclado del correo electrónico a validar
        String email = teclado.next();
        System.out.println("");

        //LLamada al método validarCorreo definido en la propia clase Principal
        validarCorreo(email);
    }

    //Método validarCorreo --> verifica que la dirección de correo cumpla los requisitos
    //Si no los cumple lanza la excepción personalizada informando del error
    public static void validarCorreo(String email) {
        //Variable booleana para indicar que el formato es correcto o no
        boolean correcto = true;

        //Bloques try-catch para evaluar y validar las excepciones (
        try {
            //Primera validación
            if (!email.contains("@")) { //Si el email no contiene @
                correcto = false;
                System.err.println("El formato de email no es válido: ");
                throw new ExcepcionEmail("El email no contiene el caracter obligatorio @");//Lanza la excepción
            }
            //Segunda validación
            if (!(email.endsWith(".com") || email.endsWith(".es"))) { //Si el email no termina en .com o ,es
                correcto = false;
                System.err.println("El formato de email no es válido: ");
                throw new ExcepcionEmail("El dominio debe ser .es o .com");//Lanza la excepción
            }
            //Tercera validación
            if (!Character.isLetter(email.charAt(0))) { //Si el email no comienza por una letra
                correcto = false;
                System.err.println("El formato de email no es válido: ");
                throw new ExcepcionEmail("El email no comienza por una letra");//Lanza la excepción
            }
        } catch (ExcepcionEmail exc) {
            //Captura la excepción en caso de ser lanzada, llama al método .getMessage de la clase ExcepcionEmail
            //que muestra el mensaje pasado anteriormente y gestionado por la clase superior Exception
            System.out.println(exc.getMessage());
        }

        if (correcto) {
            System.out.println("El email es válido");
        }
    }
}
```



> En el método **validarCorreo** gestionamos el lanzamiento de la excepción personalizada mediante bloques try-catch.

Mediante 3 condicionales (if) evaluamos si se cumple cada uno de los 3 criterios; cuando encuentra uno que no se cumple ejecuta el código de ese if lanzando un mensaje de error (formato de email no válido) en rojo (mediante **System.err.println("El formato de email no es válido: ");** y seguidamente lanza la excepción personalizada mediante un throw, pasándole al constructor como argumento un mensaje personalizado.

> En caso de que se lance una excepción será capturada por el bloque catch, lanzando por pantalla el mensaje pasado al constructor de la clase en el if anterior. Ese mensaje se pasa empleando el método **.getMessage()** que la clase **ExcepcionEmail** hereda de la clase **Exception**.



> Email que no contenga @:

```
run:
--Introduzca un email con formato válido--
-----
- Contendrá @
- Dominios válidos: .com y .es
- Comenzará por una letra

Email: rwigrvr.es

El formato de email no es válido:
El email no contiene el caracter obligatorio @
```

> Email con dominio no válido:

```
run:
--Introduzca un email con formato válido--
-----
- Contendrá @
- Dominios válidos: .com y .es
- Comenzará por una letra

Email: fran@gmail.rr

El formato de email no es válido:

El dominio debe ser .es o .com
```

> Email que no comienza por una letra:

```
run:
--Introduzca un email con formato válido--
-----
- Contendrá @
- Dominios válidos: .com y .es
- Comenzará por una letra

Email: 3fran79@hotmail.es

El formato de email no es válido:
El email no comienza por una letra
```

> Email válido:

```
run:
--Introduzca un email con formato válido--
-----
- Contendrá @
- Dominios válidos: .com y .es
- Comenzará por una letra

Email: fcojgarciafco@gmail.com

El email es válido
```



> ExcepcionEmail.java

```
package reto3;

public class ExcepcionEmail extends Exception {

    //Constructores

    //1 --> Por defecto: construye pasando un mensaje predefinido,
    //por defecto, a la clase padre (Exception)
    public ExcepcionEmail() {
        super("El correo electrónico introducido no es válido");
    }

    //2 --> Construye pasando un mensaje específico
    //recibido en una variable tipo String
    public ExcepcionEmail(String mensaje) {
        super(mensaje);
    }
}
```



> Principal.java

```
package reto3;

import java.util.Scanner;

public class Principal {

    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        System.out.println("--Introduzca un email con formato válido--");
        System.out.println("-----");
        System.out.println("""
            - Contendrá @
            - Dominios válidos: .com y .es
            - Comenzará por una letra
            """);
        System.out.println("");
        System.out.print("Email: ");

        //Toma por teclado del correo electrónico a validar
        String email = teclado.next();
        System.out.println("");

        //Llamada al método validarCorreo definido en la propia clase Principal
        validarCorreo(email);
    }

    //Método validarCorreo --> verifica que la dirección de correo cumpla los requisitos
    //Si no los cumple lanza la excepción personalizada informando del error
    public static void validarCorreo(String email) {
        //Variable booleana para indicar que el formato es correcto o no
        boolean correcto = true;

        //Bloques try-catch para evaluar y validar las excepciones (
        try {
            //Primera validación
            if (!email.contains("@")) { //Si el email no contiene @
                correcto = false;
                System.err.println("El formato de email no es válido: ");
                throw new ExcepcionEmail("El email no contiene el caracter obligatorio @");//Lanza la excepción
            }
            //Segunda validación
            if (!(email.endsWith(".com") || email.endsWith(".es"))) { //Si el email no termina en .com o .es
                correcto = false;
                System.err.println("El formato de email no es válido: ");
                throw new ExcepcionEmail("El dominio debe ser .es o .com");//Lanza la excepción
            }
            //Tercera validación
            if (!Character.isLetter(email.charAt(0))) { //Si el email no comienza por una letra
                correcto = false;
                System.err.println("El formato de email no es válido: ");
                throw new ExcepcionEmail("El email no comienza por una letra");//Lanza la excepción
            }
        }
    }
}
```




```
} catch (ExcepcionEmail exc) {  
    //Captura la excepción en caso de ser lanzada, llama al método .getMessage de la clase ExcepcionEmail  
    //que muestra el mensaje pasado anteriormente y gestionado por la clase superior Exception  
    System.out.println(exc.getMessage());  
}  
  
if (correcto) {  
    System.out.println("El email es válido");  
}  
}  
}
```